

Parallel and Distributed Computing

Lock

Kun Suo

Computer Science, Kennesaw State University

Race condition

- A race condition occurs when two or more threads **access** shared data and they try to **change** it at the **same time**.
- The **order** in which the threads attempt to access the shared data makes the results unpredictable

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int counter = 0;

void *compute()
{
    int i = 0;
    while (i < 100) {
        counter = counter + 1;
        i++;
    }
    printf("Counter value: %d\n", counter);
}

int main()
{
    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, compute, (void *)&thread1);
    pthread_create(&thread2, NULL, compute, (void *)&thread2);

    pthread_exit(NULL);
    exit(0);
}
```

Race condition occurs for variable counter

```
pi@raspberrypi ~/Downloads> ./race_condition.o
Counter value: 100
Counter value: 200
```

Seem nothing wrong?

Race condition example

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int counter = 0;

void *compute()
{
    int i = 0;
    while (i < 10000) {
        counter = counter + 1;
        i++;
    }
    printf("Counter value: %d\n", counter);
}

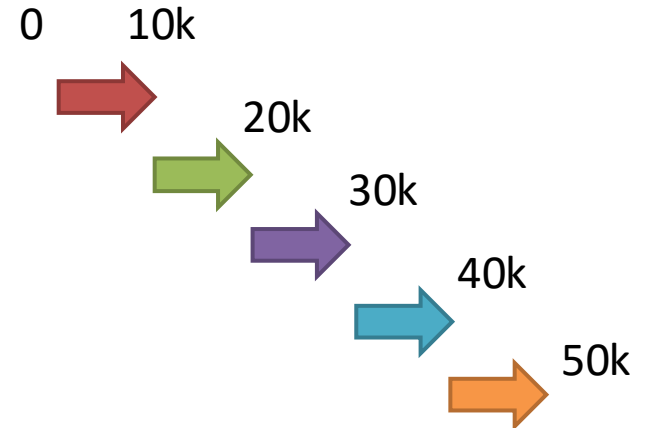
int main()
{
    pthread_t thread1, thread2, thread3, thread4, thread5;

    pthread_create(&thread1, NULL, compute, (void *)&thread1);
    pthread_create(&thread2, NULL, compute, (void *)&thread2);
    pthread_create(&thread3, NULL, compute, (void *)&thread3);
    pthread_create(&thread4, NULL, compute, (void *)&thread4);
    pthread_create(&thread5, NULL, compute, (void *)&thread5);

    pthread_exit(NULL);
    exit(0);
}
```

Increase the loop number

Add more threads

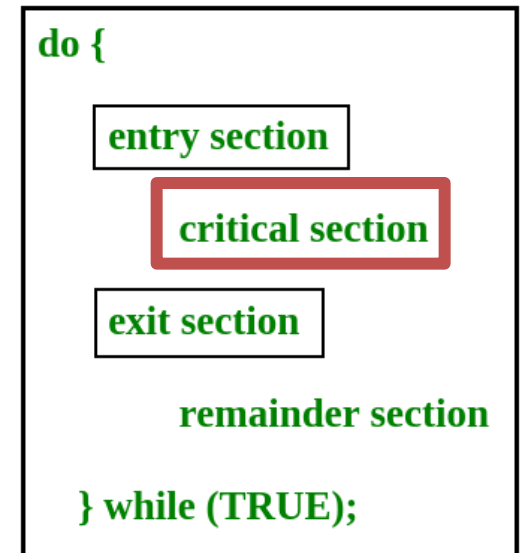


```
pi@raspberrypi ~/Downloads> ./race_condition.o
Counter value: 14467
Counter value: 10410
Counter value: 12080
Counter value: 22745
Counter value: 32725
```

Weird results!

Critical section

- A section of code in a concurrent task that **modifies or accesses** a resource shared with another task.
- Examples
 - A piece of code that reads from or writes to a shared memory region
 - Or a code that modifies or traverses a shared linked list.



Critical section example

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int counter = 0;

void *compute()
{
    int i = 0;
    while (i < 100) {
        counter = counter + 1;
        i++;
    }
    printf("Counter value: %d\n", counter);
}

int main()
{
    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, compute, (void *)&thread1);
    pthread_create(&thread2, NULL, compute, (void *)&thread2);

    pthread_exit(NULL);
    exit(0);
}
```

Critical section: All threads read and write the shared counter



Critical section vs. Race condition

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int counter = 0;

void *compute()
{
    int i = 0;
    while (i < 100) {
        counter = counter + 1;
        i++;
    }
    printf("Counter value: %d\n", counter);
}

int main()
{
    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, compute, (void *)&thread1);
    pthread_create(&thread2, NULL, compute, (void *)&thread2);

    pthread_exit(NULL);
    exit(0);
}
```

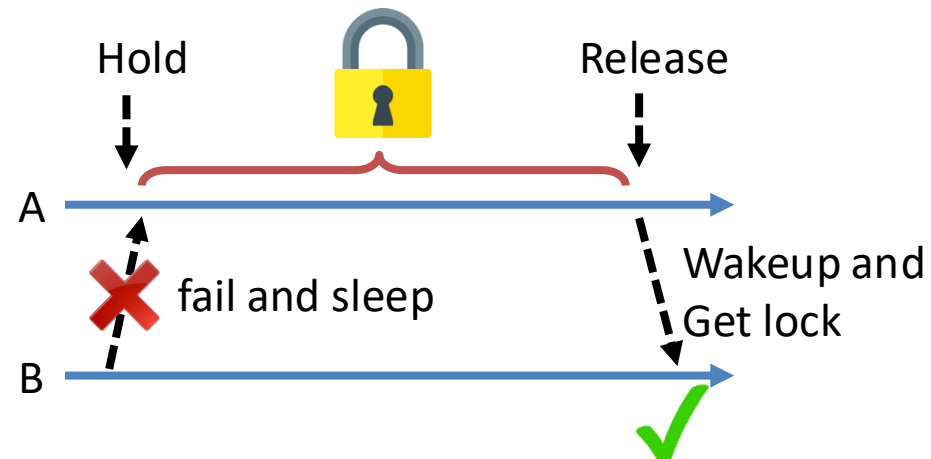
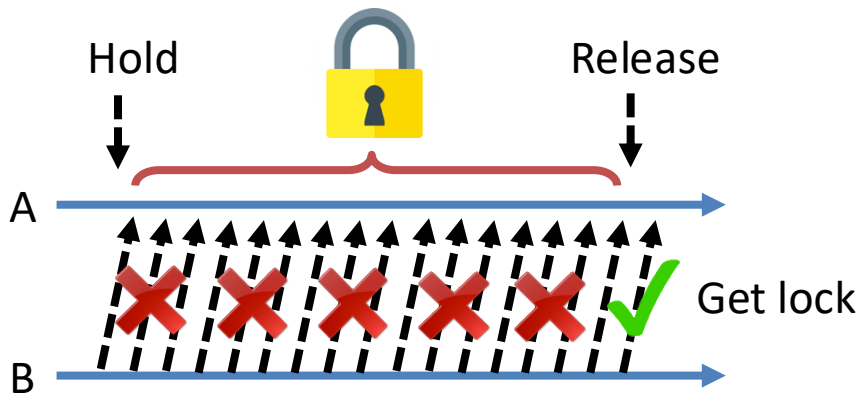
Critical section is where the race condition happens.

When multiple threads visit the critical section, race condition problem appears!



Lock (mutual exclusion)

- A lock (mutual exclusion) is a synchronization mechanism for enforcing limits on access to a resource in an environment where there are many threads of execution
- Types of mutual mechanism:
 - Busy-waiting, e.g., spinlock
 - Sleep and wakeup

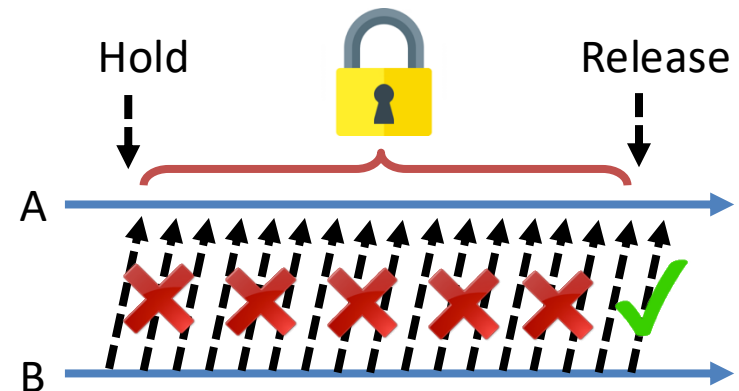


1, Spinlock: A busy-waiting lock implementation

- Don't block. Instead, constantly poll the lock for availability.
- Usage: small critical region
- Advantage
 - Very efficient with short critical sections
 - if you expect a lock to be released quickly
- Disadvantage
 - Doesn't yield the CPU and burns CPU cycles
 - Bad if critical sections are long.
 - Efficient only if machine has multiple CPUs.
 - Counterproductive on uniprocessor machines

```
while (lock is unavailable)
    continue; // try again
return success;
```

```
SpinLock(resource);
Execute Critical Section;
SpinUnlock(resource);
```



Without Spinlock example

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

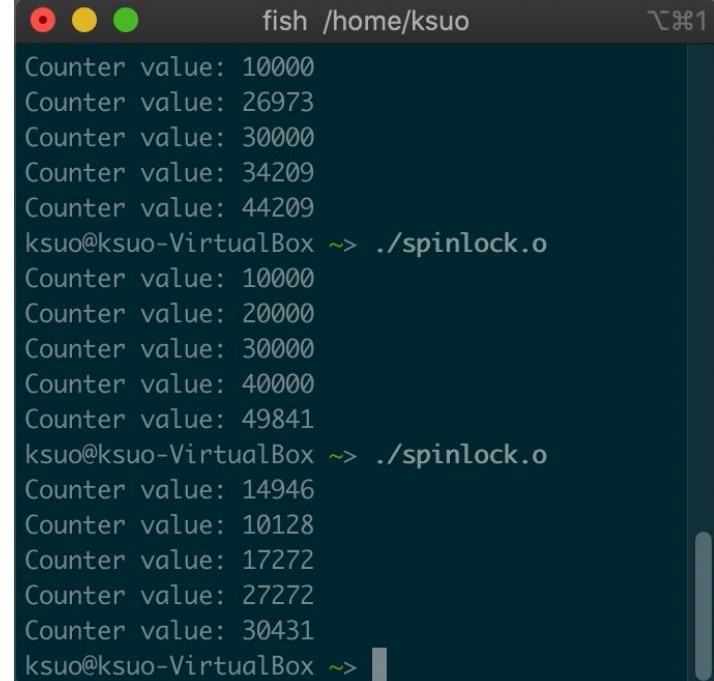
int counter = 0;
static pthread_spinlock_t splock;

void *compute()
{
    int i = 0;
    while (i < 10000) {
        counter = counter + 1;
        i++;
    }
    printf("Counter value: %d\n", counter);
}

int main()
{
    pthread_t thread1, thread2, thread3, thread4, thread5;

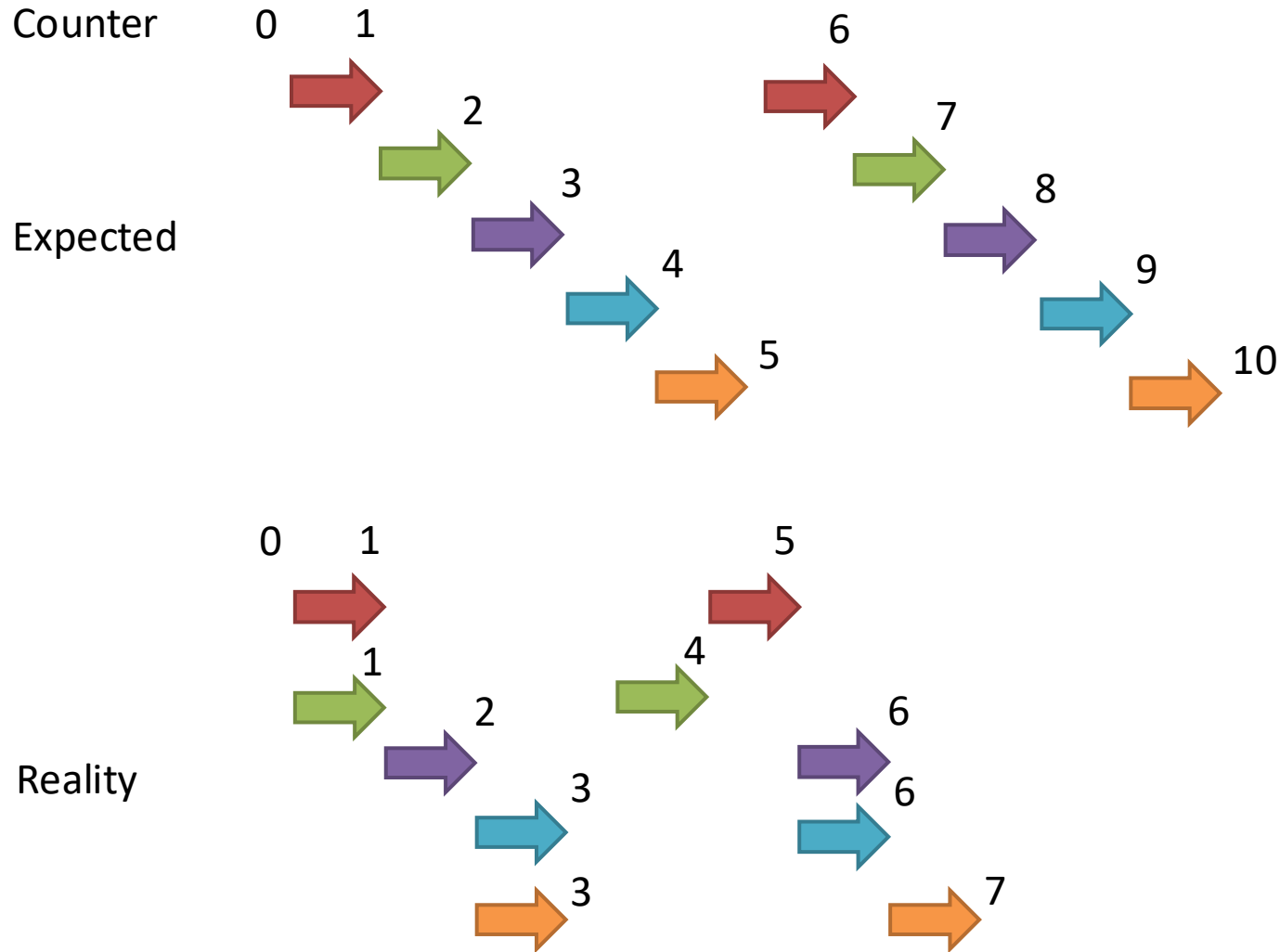
    pthread_create(&thread1, NULL, compute, (void *)&thread1);
    pthread_create(&thread2, NULL, compute, (void *)&thread2);
    pthread_create(&thread3, NULL, compute, (void *)&thread3);
    pthread_create(&thread4, NULL, compute, (void *)&thread4);
    pthread_create(&thread5, NULL, compute, (void *)&thread5);

    pthread_exit(NULL);
    exit(0);
}
```



```
fish /home/ksuo
Counter value: 10000
Counter value: 26973
Counter value: 30000
Counter value: 34209
Counter value: 44209
ksuo@ksuo-VirtualBox ~> ./spinlock.o
Counter value: 10000
Counter value: 20000
Counter value: 30000
Counter value: 40000
Counter value: 49841
ksuo@ksuo-VirtualBox ~> ./spinlock.o
Counter value: 14946
Counter value: 10128
Counter value: 17272
Counter value: 27272
Counter value: 30431
ksuo@ksuo-VirtualBox ~> 
```

Without Spinlock example



Spinlock example

[spinlock.c](#)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int counter = 0;
static pthread_spinlock_t splock;

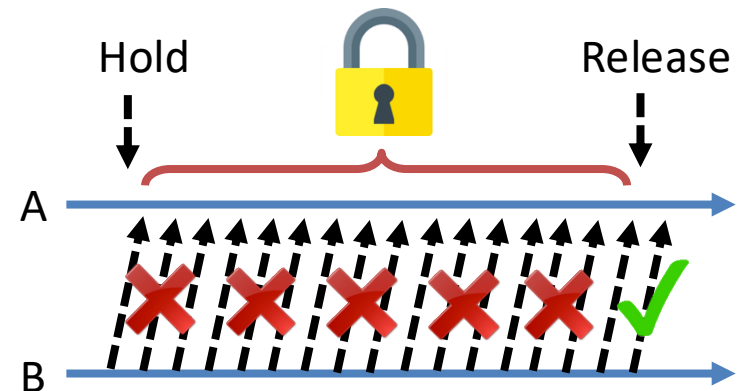
void *compute()
{
    int i = 0;
    pthread_spin_lock(&splock);
    while (i < 10000) {
        counter = counter + 1;
        i++;
    }
    printf("Counter value: %d\n", counter);
    pthread_spin_unlock(&splock);
}

int main()
{
    pthread_t thread1, thread2, thread3, thread4, thread5;

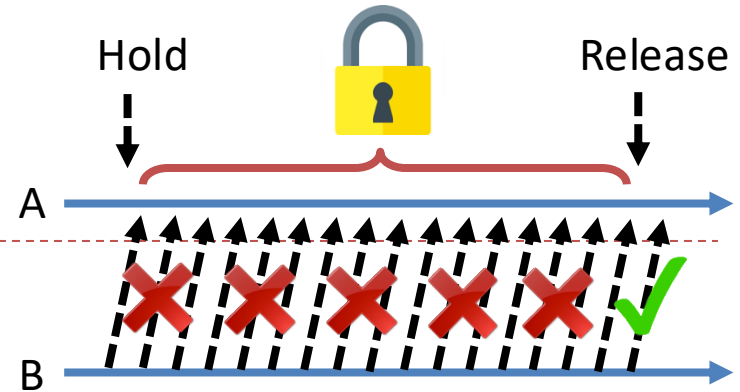
    pthread_create(&thread1, NULL, compute, (void *)&thread1);
    pthread_create(&thread2, NULL, compute, (void *)&thread2);
    pthread_create(&thread3, NULL, compute, (void *)&thread3);
    pthread_create(&thread4, NULL, compute, (void *)&thread4);
    pthread_create(&thread5, NULL, compute, (void *)&thread5);

    pthread_exit(NULL);
    exit(0);
}
```

```
ksuo@ksuo-VirtualBox ~/Desktop> ./spinlock.o
Counter value: 10000
Counter value: 20000
Counter value: 30000
Counter value: 40000
Counter value: 50000
```



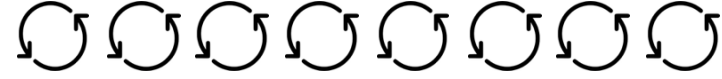
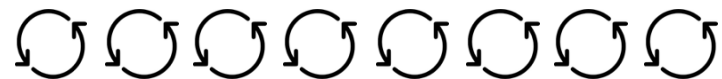
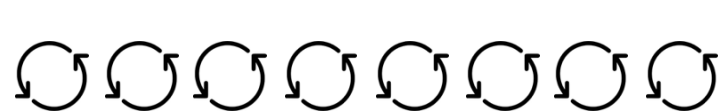
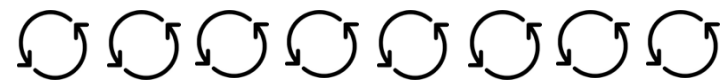
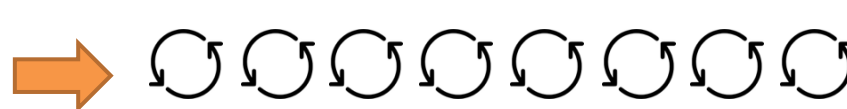
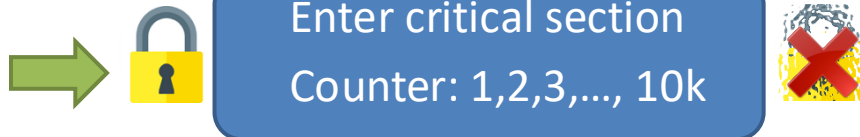
Spinlock example



Counter
is 0

Counter
is 10k

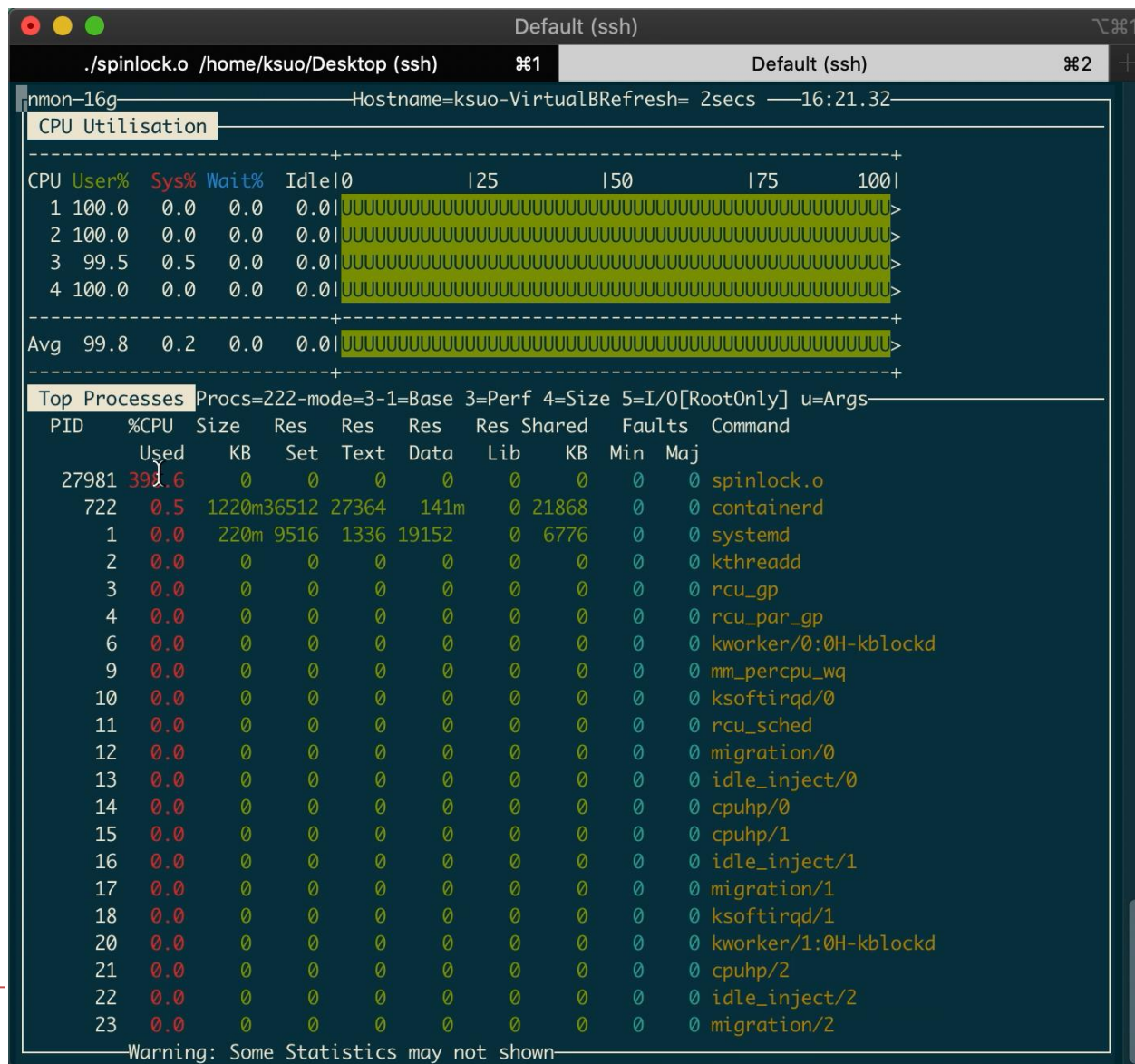
Counter
is 20k



Enter critical section
Counter: 10k+1,...,20k



Spinlock example: CPU utilization



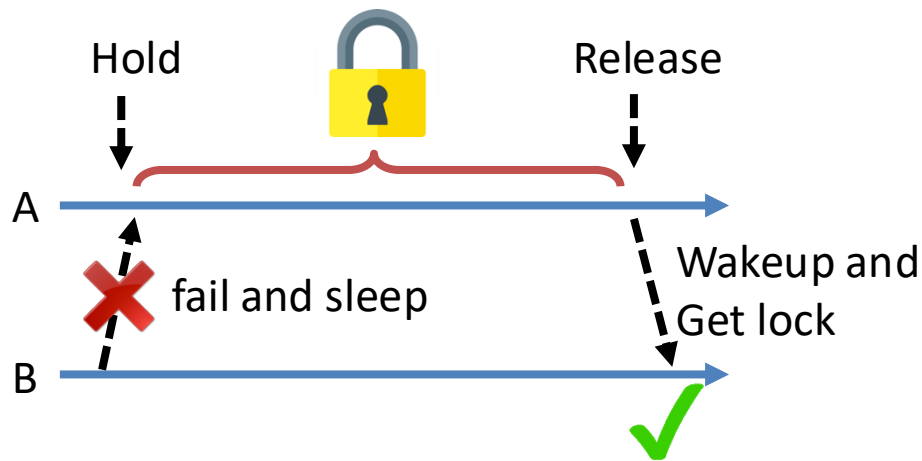
Other mutual exclusion similar as busy waiting (spinlock)

- Disabling interrupts:
 - OS technique, not users'
- Lock variables:
 - Test-and-set lock (TSL) is a two-step process, not atomic
- Peterson's algorithm
 - Does not need atomic operation and mainly used in user space application



2, Mutex lock: A sleep-and-wakeup lock implementation

- A variable that can be in one of two states: unlocked or locked
- Mutex is used as a LOCK around critical sections



Example:

```
Lock(mutex)
Critical Section...
Unlock(mutex)
```

Pro:

Better cpu utilization

Con:

Overhead on entering sleep or wake up

Not suited for short duration of lock acquisition

Mutex lock example

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int counter = 0;
static pthread_mutex_t mlock;

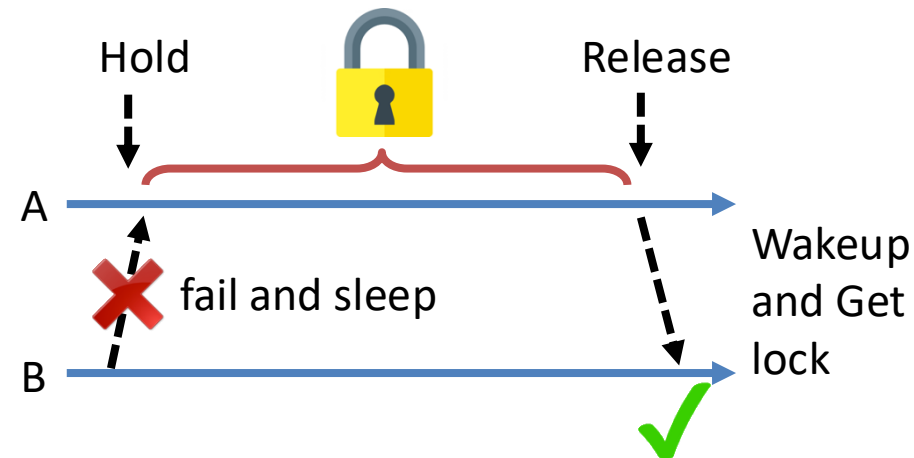
void *compute()
{
    int i = 0;
    pthread_mutex_lock(&mlock);
    while (i < 10000) {
        counter = counter + 1;
        i++;
    }
    printf("Counter value: %d\n", counter);
    pthread_mutex_unlock(&mlock);
}

int main()
{
    pthread_t thread1, thread2, thread3, thread4, thread5;

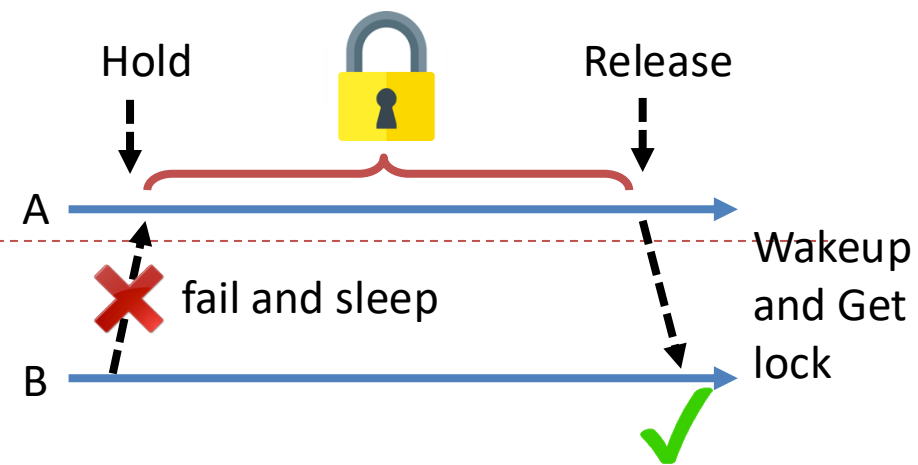
    pthread_create(&thread1, NULL, compute, (void *)&thread1);
    pthread_create(&thread2, NULL, compute, (void *)&thread2);
    pthread_create(&thread3, NULL, compute, (void *)&thread3);
    pthread_create(&thread4, NULL, compute, (void *)&thread4);
    pthread_create(&thread5, NULL, compute, (void *)&thread5);

    pthread_exit(NULL);
    exit(0);
}
```

```
pi@raspberrypi ~/Downloads> ./mutexlock.o
Counter value: 10000
Counter value: 20000
Counter value: 30000
Counter value: 40000
Counter value: 50000
```



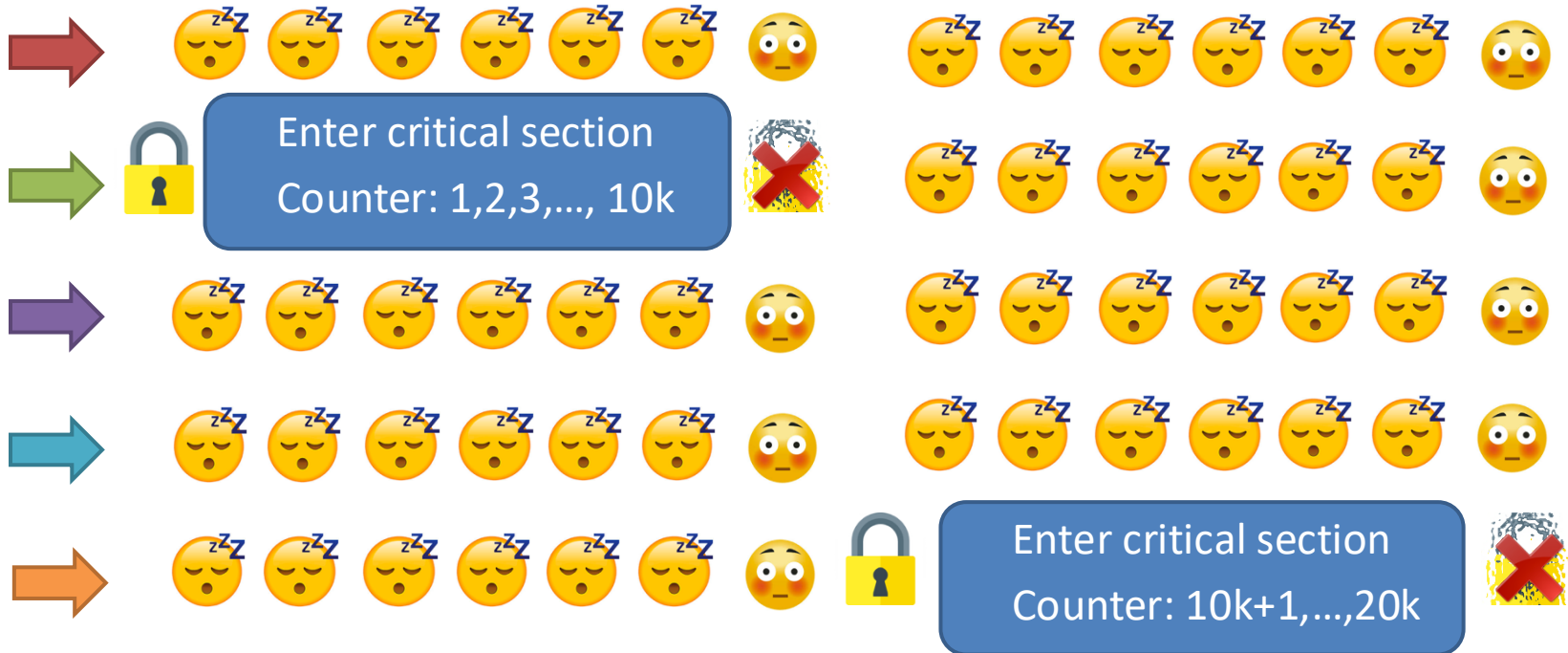
Mutex example



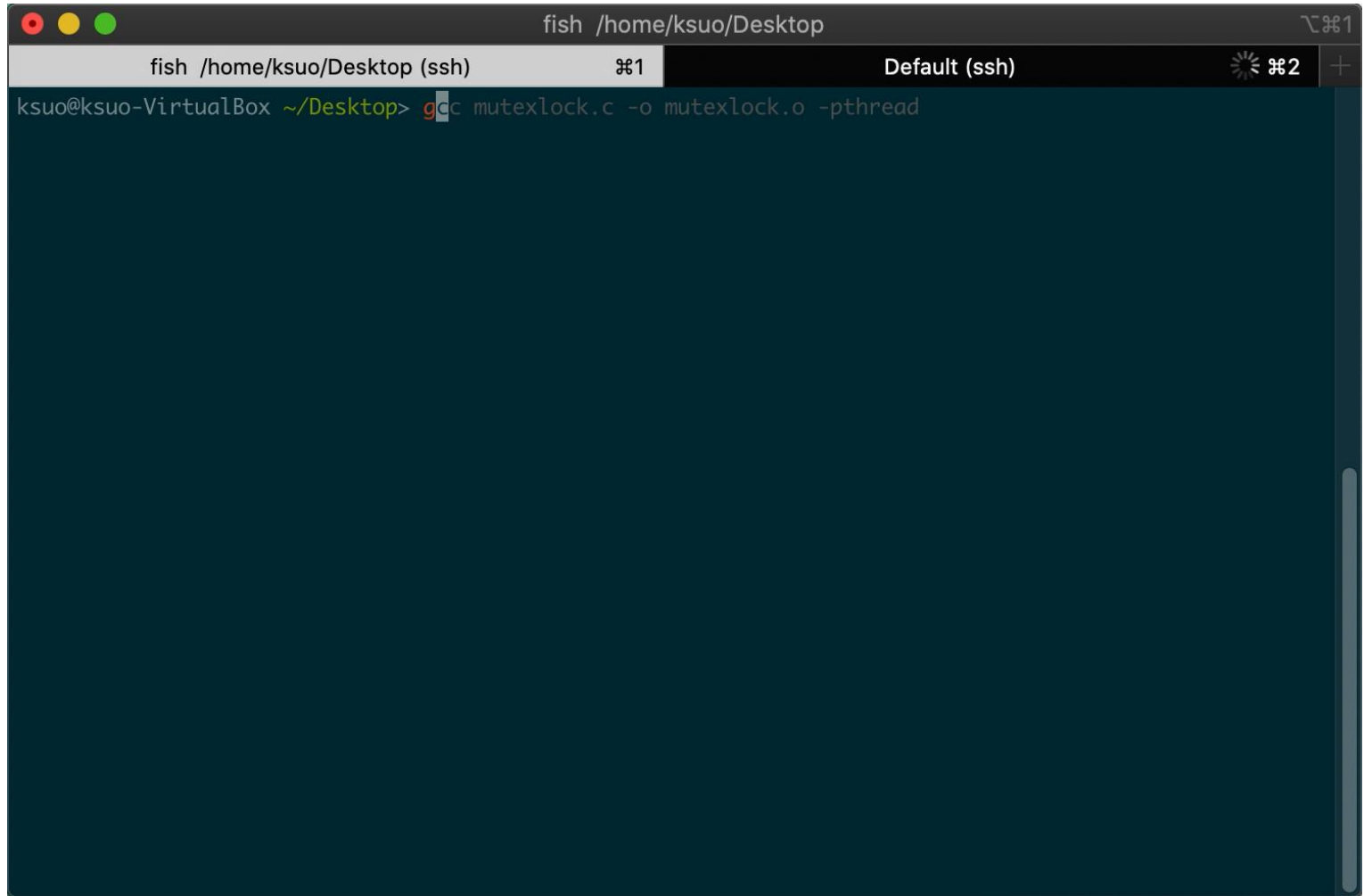
Counter
is 0

Counter
is 10k

Counter
is 20k



Mutex lock CPU utilization



A terminal window with a dark background and light text. The window title bar shows 'fish /home/ksuo/Desktop' and a window icon. Below the title bar, there are two tabs: 'fish /home/ksuo/Desktop (ssh)' and 'Default (ssh)'. The first tab is active and shows the prompt 'ksuo@ksuo-VirtualBox ~/Desktop>'. The command 'gcc mutexlock.c -o mutexlock.o -pthread' is entered at the prompt. The second tab is inactive and shows a loading icon and the text 'Default (ssh)'. The terminal window has a vertical scrollbar on the right side.

```
fish /home/ksuo/Desktop
fish /home/ksuo/Desktop (ssh)  1
Default (ssh)  2
ksuo@ksuo-VirtualBox ~/Desktop> gcc mutexlock.c -o mutexlock.o -pthread
```



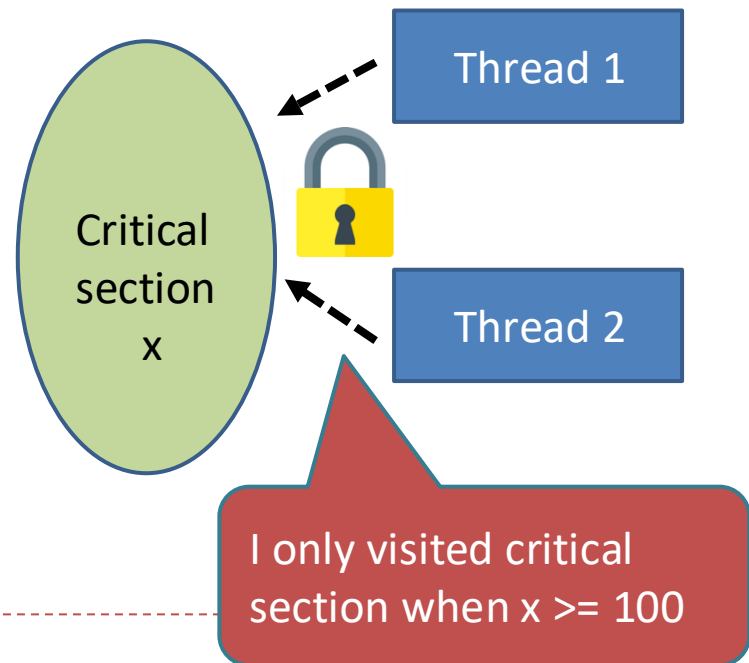
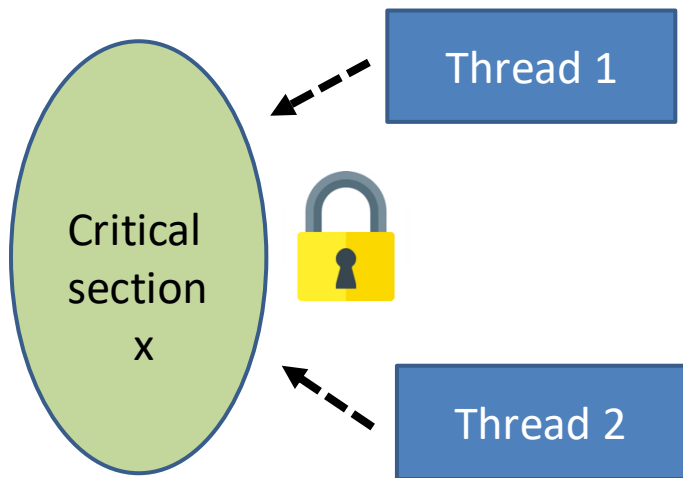
Busy waiting lock vs Sleep wake up lock

	Mechanism	Use case	Implementation	Other examples
Busy waiting lock	constantly poll the lock for availability	When the waiting time is short	Spin lock	Disabling interrupts; Lock variables; Peterson's algorithm
Sleep wake up lock	Sleep if lock not available; wake up if available	When the waiting time is long	Mutex lock	Semaphore



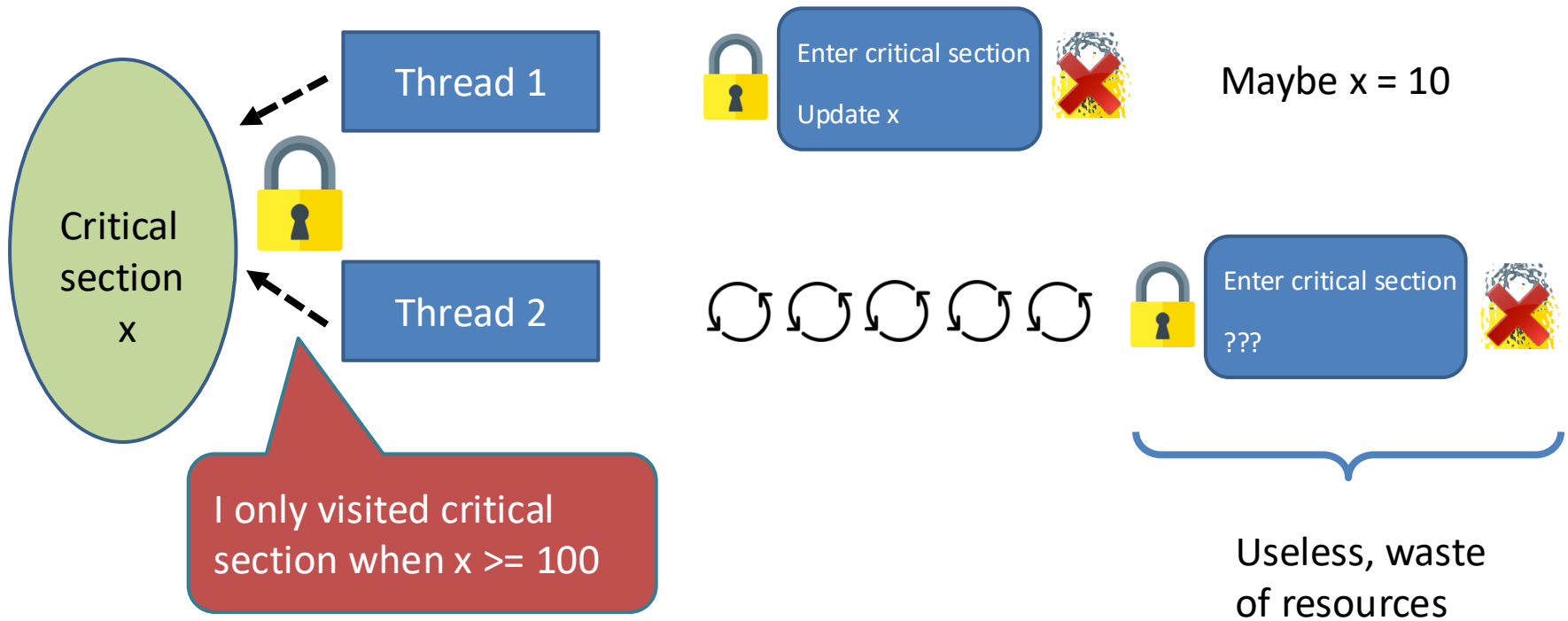
3. Mutex lock with conditions

- Mutex locks solve the competition problem of multiple threads accessing the same global variable under the shared memory space. (**without conditions**)
- How about competition **with condition** variables?



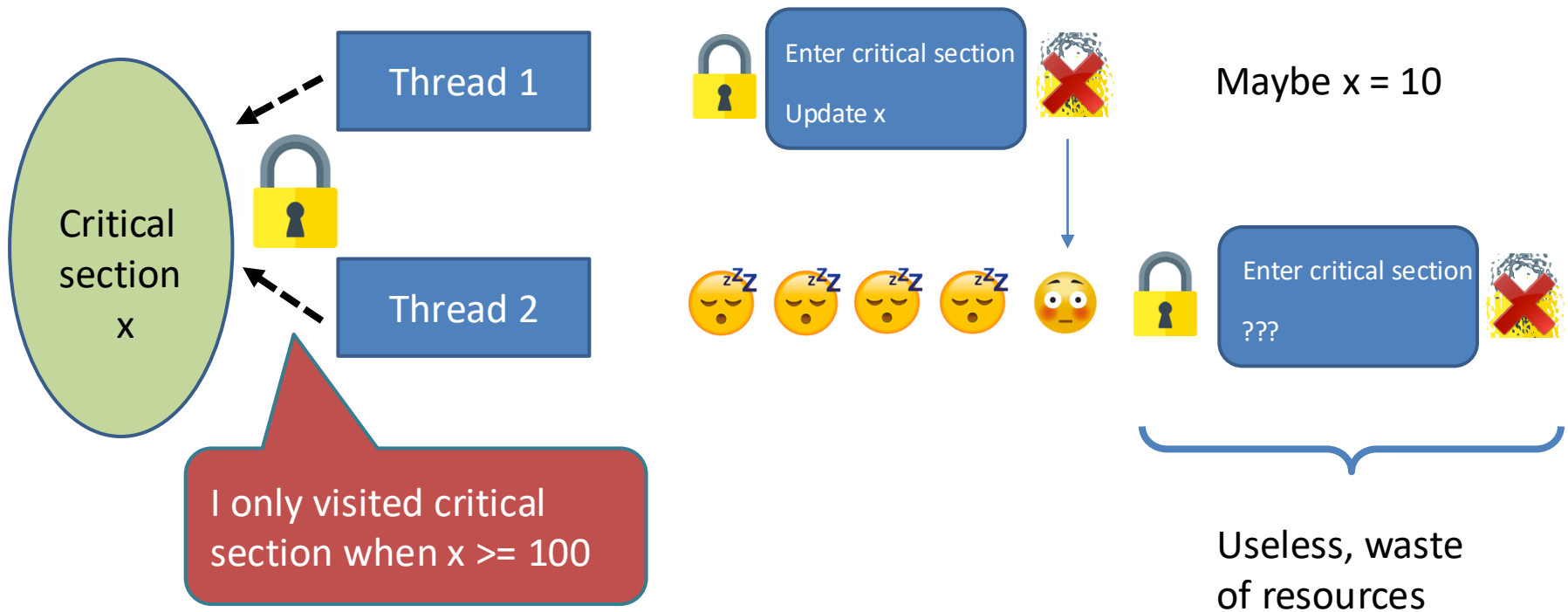
3. Mutex lock with conditions

- Can we use busy-waiting lock?



3. Mutex lock with conditions

- Can we use sleep-and-wakeup lock?



Mutex lock with conditions example

- How about competition **with condition** variables?
 - Example: T1: increase x every time;
 - T2: when x is larger than 99, then set x to 0;

```
//thread 1:  
  
while(true)  
{  
  
    iCount++;  
  
}
```

```
//thread 2:  
while(true)  
{  
  
    if(iCount >= 100)  
    {  
        iCount = 0;  
    }  
  
}
```

T1 and T2 compete
for variable iCount!



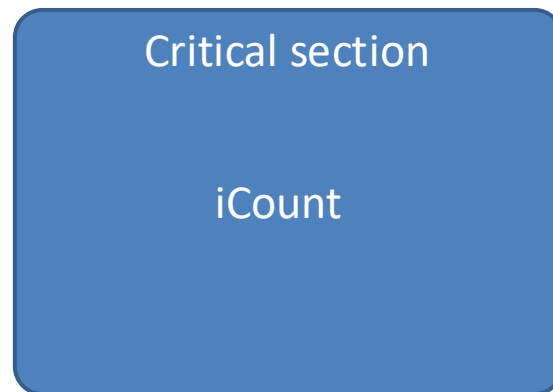
Mutex lock with conditions example

```
//thread 1:  
  
while(true)  
{  
  
    iCount++;  
  
}
```

```
//thread 2:  
while(true)  
{  
  
    if(iCount >= 100)  
    {  
        iCount = 0;  
    }  
  
}
```

Thread 1 →

Thread 2 →



Mutex lock with conditions

- How about competition **with condition** variables?
 - Example: T1: increase x every time;
 - T2: when x is larger than 99, then set x to 0;

```
//thread 1:  
  
while(true)  
{  
    pthread_mutex_lock(&mutex);  
    iCount++;  
    pthread_mutex_unlock(&mutex);  
}
```

```
//thread 2:  
while(true)  
{  
    pthread_mutex_lock(&mutex);  
    if(iCount >= 100)  
    {  
        iCount = 0;  
    }  
    pthread_mutex_unlock(&mutex);  
}
```

T2 needs to:
lock;
determine;
unlock;
every time to check

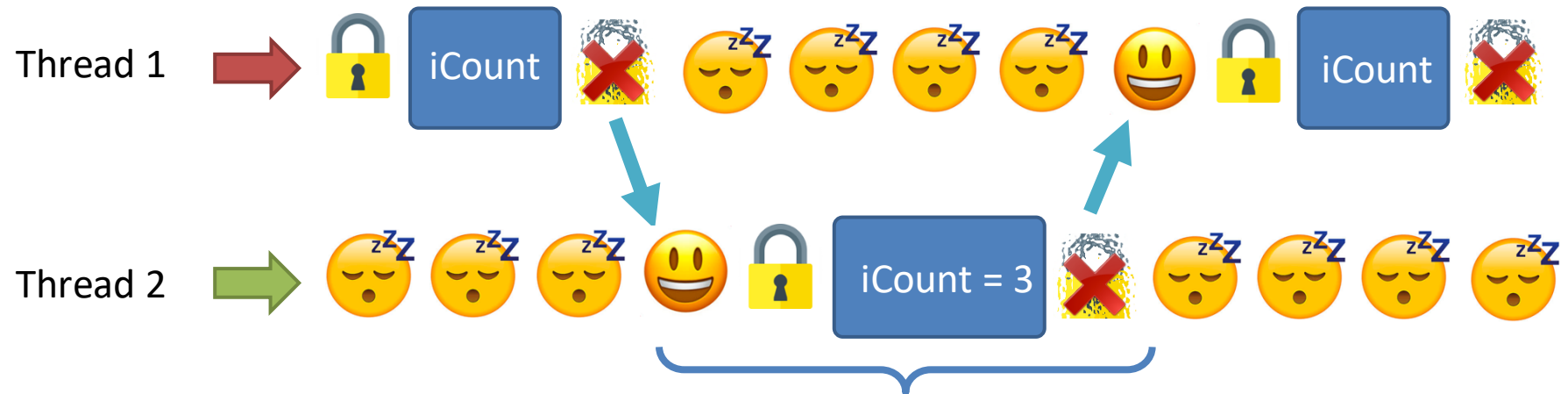


Mutex lock with conditions

```
//thread 1:  
  
while(true)  
{  
    pthread_mutex_lock(&mutex);  
    iCount++;  
    pthread_mutex_unlock(&mutex);  
}
```

```
//thread 2:  
while(true)  
{  
    pthread_mutex_lock(&mutex);  
    if(iCount >= 100)  
    {  
        iCount = 0;  
    }  
    pthread_mutex_unlock(&mutex);  
}
```

Critical section



```
int iCount = 0;
static pthread_mutex_t mlock;
```

Examples

```
void *thread1_work(void *id) {
    long tid = (long)id;
    while (1) {
        pthread_mutex_lock(&mlock);
        iCount++;
        printf("thread: %ld iCount: %d\n", tid, iCount);
        pthread_mutex_unlock(&mlock);
        sleep(1);
    }
}

void *thread2_work(void *id) {
    long tid = (long)id;
    while (1) {
        pthread_mutex_lock(&mlock);
        if (iCount >= 100)
            iCount = 0;
        printf("thread: %ld iCount: %d\n", tid, iCount);
        pthread_mutex_unlock(&mlock);
        sleep(1);
    }
}

int main() {
    pthread_t thread1, thread2;
    int id1=1, id2=2;
    if (pthread_mutex_init(&mlock, NULL) != 0) {
        printf("mutex init failed\n");
        return 1;
    }

    pthread_create(&thread1, NULL, thread1_work, (void *)(&id1));
    pthread_create(&thread2, NULL, thread2_work, (void *)(&id2));

    pthread_exit(NULL);
    pthread_mutex_destroy(&mlock);
    exit(0);
}
```

lock wo condition.c

```
//thread 1:
```

```
while(true)
```

```
{
```

```
    pthread_mutex_lock(&mutex);
```

```
    iCount++;
```

```
    pthread_mutex_unlock(&mutex);
```

```
}
```

```
//thread 2:
```

```
while(true)
```

```
{
```

```
    pthread_mutex_lock(&mutex);
```

```
    if(iCount >= 100)
```

```
    {
```

```
        iCount = 0;
```

```
    }
```

```
    pthread_mutex_unlock(&mutex);
```

```
}
```

Pthread_cond_signal and Pthread_cond_wait

Release the lock
Sleep here until condition is reached

- Pthread_cond_wait(&condition, &lock)



- Pthread_cond_signal(&condition)

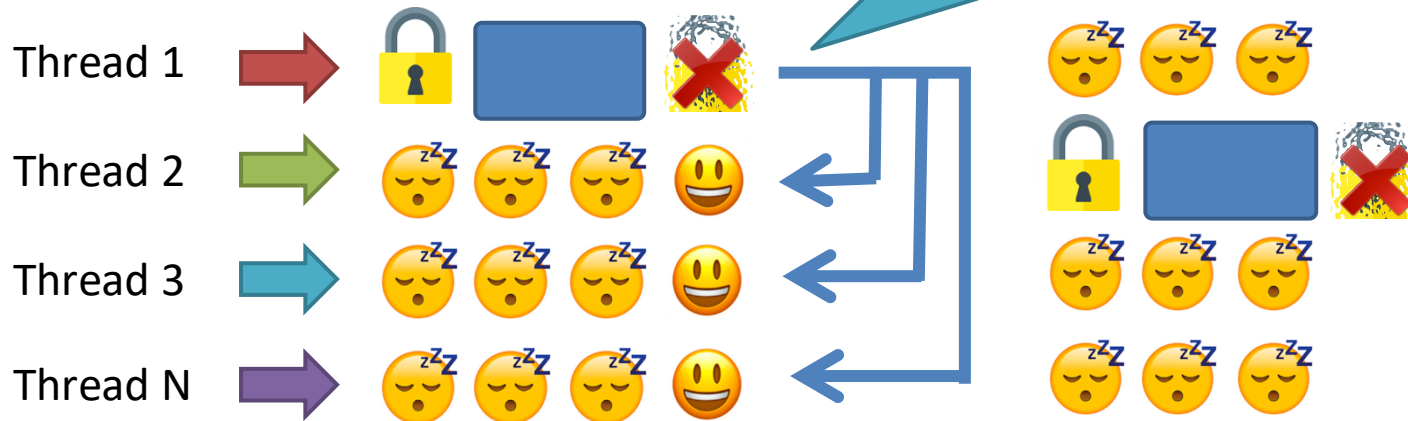


When condition is reached,
notify all threads waiting for it

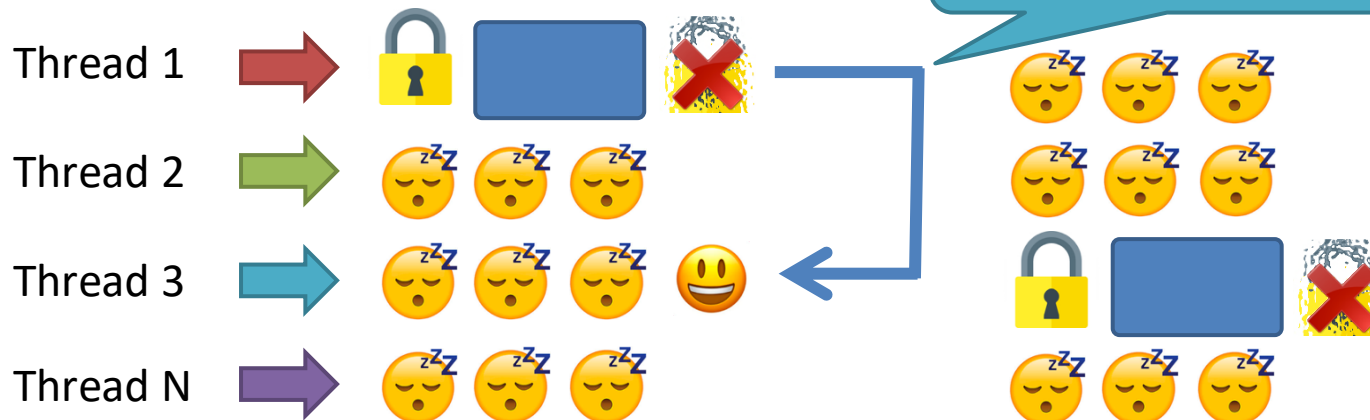


Pthread_cond_signal/Pthread_cond_wait v.s. Pthread_mutex_lock/unlock

All threads sleeping will be waked up when lock is released



Only wake up the threads with conditions



Condition variable

- How about competition **with condition** variables?
 - Example: T1: increase x every time;
 - T2: when x is larger than 99, then set x to 0;

When T2 executes here:
❑ release mutex
❑ blocked here
❑ when waked, get mutex and execute

```
//thread1 :  
while(true)  
{  
    pthread_mutex_lock(&mutex);  
    iCount++;  
    pthread_mutex_unlock(&mutex);  
  
    pthread_mutex_lock(&mutex);  
    if(iCount >= 100)  
    {  
        pthread_cond_signal(&cond);  
    }  
    pthread_mutex_unlock(&mutex);  
}
```

```
//thread2:  
while(1)  
{  
    pthread_mutex_lock(&mutex);  
    while(iCount < 100)  
    {  
        pthread_cond_wait(&cond, &mutex);  
    }  
    printf("iCount >= 100\r\n");  
    iCount = 0;  
    pthread_mutex_unlock(&mutex);  
}
```

```
//thread1 :
while(true)
{
    pthread_mutex_lock(&mutex);
    iCount++;
    pthread_mutex_unlock(&mutex);

    pthread_mutex_lock(&mutex);
    if(iCount >= 100)
    {
        pthread_cond_signal(&cond);
    }
    pthread_mutex_unlock(&mutex);
}
```

1. Get the 
2. release the 
3. Get the 
4. release the 

```
//thread2:
while(1)
{
    pthread_mutex_lock(&mutex);
    while(iCount < 100)
    {
        pthread_cond_wait(&cond, &mutex);
    }
    printf("iCount >= 100\r\n");
    iCount = 0;
    pthread_mutex_unlock(&mutex);
}
```

1. Get the 
2. release the 
3. Get 
4. release the 

3. Wake up



T2 needs to:
lock;
determine;
unlock;
every time to check

```
//thread 1:
while(true)
{
    pthread_mutex_lock(&mutex);
    iCount++;
    pthread_mutex_unlock(&mutex);
}
```

1. 
2. Release the 

```
// thread 2:
while(true)
{
    pthread_mutex_lock(&mutex);
    if(iCount >= 100)
    {
        iCount = 0;
    }
    pthread_mutex_unlock(&mutex);
}
```

Condition variable example

[lock w condition.c](#)

```
int iCount = 0;
static pthread_mutex_t mlock;
static pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

void *thread1_work(void *id) {
    long tid = (long)id;
    while (1) {
        pthread_mutex_lock(&mlock);
        iCount++;
        printf("thread: %ld iCount: %d\n", tid, iCount);
        pthread_mutex_unlock(&mlock);

        pthread_mutex_lock(&mlock);
        if (iCount >= 100) {
            pthread_cond_signal(&cond);
            printf("thread: %ld iCount: %d\n", tid, iCount);
        }
        pthread_mutex_unlock(&mlock);
        sleep(1);
    }
}

void *thread2_work(void *id) {
    long tid = (long)id;
    while (1) {
        pthread_mutex_lock(&mlock);
        if (iCount < 100) {
            pthread_cond_wait(&cond, &mlock);
        }
        iCount = 0;
        printf("thread: %ld iCount: %d\n", tid, iCount);
        pthread_mutex_unlock(&mlock);
        sleep(1);
    }
}
```

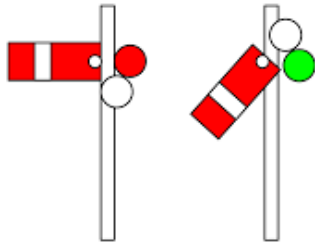
```
//thread1 :
while(true)
{
    pthread_mutex_lock(&mutex);
    iCount++;
    pthread_mutex_unlock(&mutex);

    pthread_mutex_lock(&mutex);
    if(iCount >= 100)
    {
        pthread_cond_signal(&cond);
    }
    pthread_mutex_unlock(&mutex);
}
```

```
//thread2:
while(1)
{
    pthread_mutex_lock(&mutex);
    while(iCount < 100)
    {
        pthread_cond_wait(&cond, &mutex);
    }
    printf("iCount >= 100\r\n");
    iCount = 0;
    pthread_mutex_unlock(&mutex);
}
```

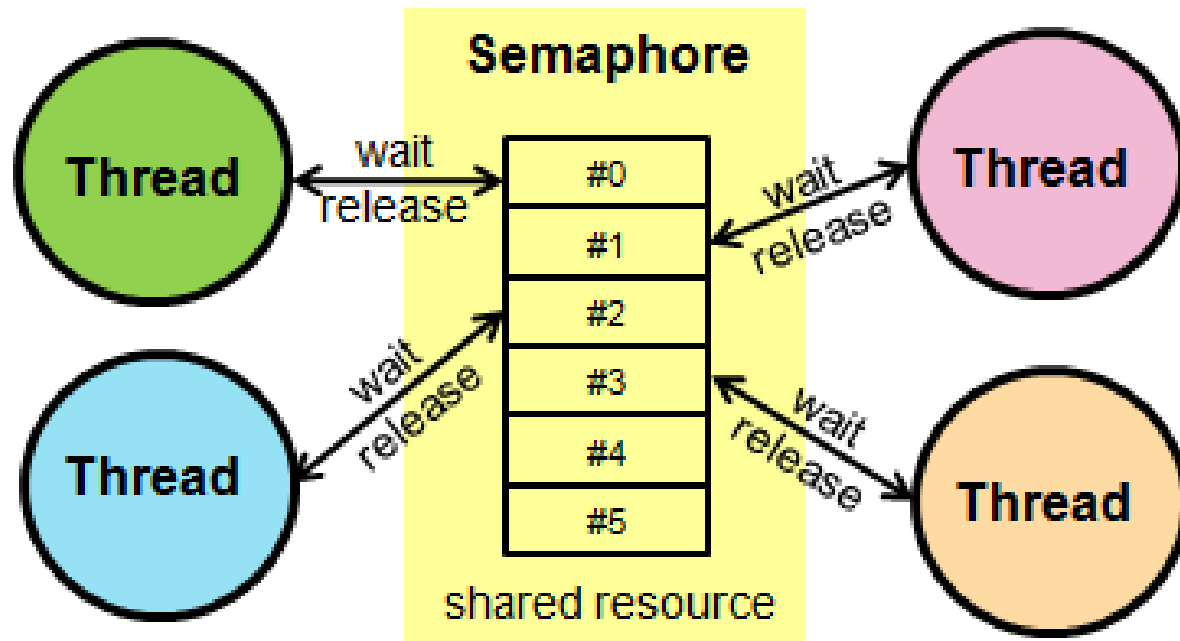
4. Semaphore

a system of sending messages by holding the arms or two flags in certain positions



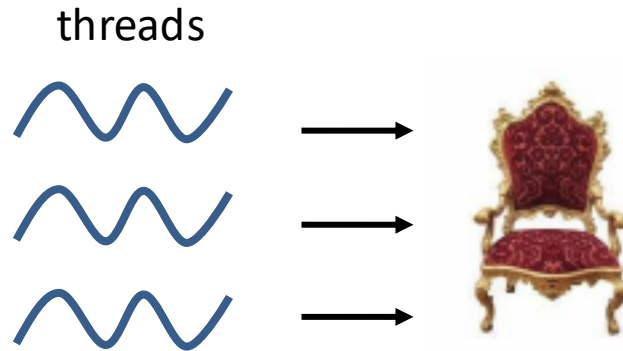
4. Semaphore

- Semaphore is a variable used to control access to shared resources by multiple processes/threads



Mutex lock and Semaphore

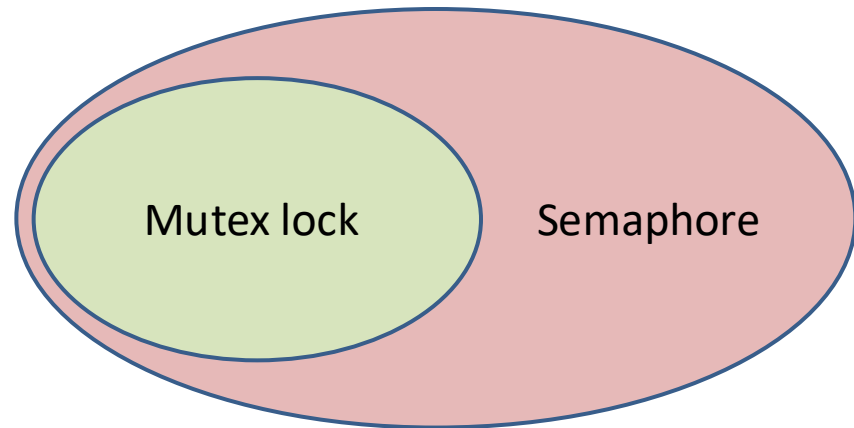
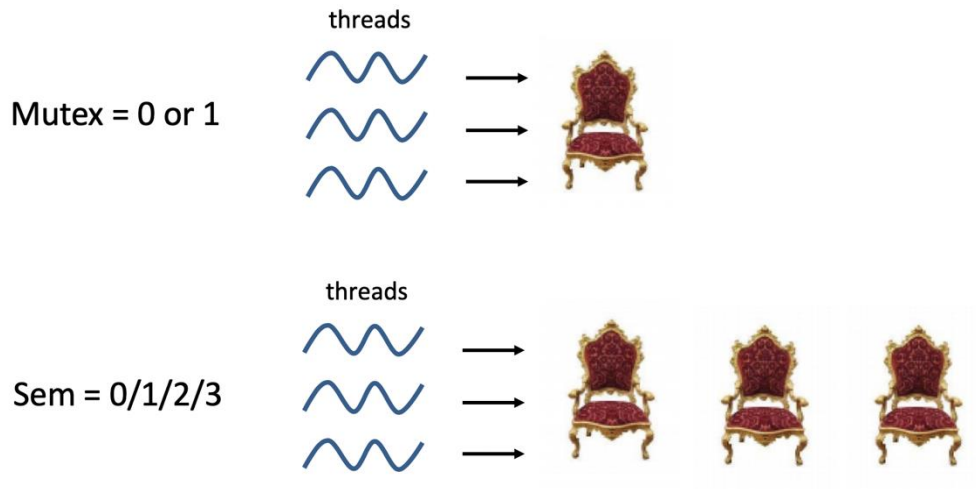
Mutex = 0 or 1



Sem = 0/1/2/3



Mutex lock and Semaphore

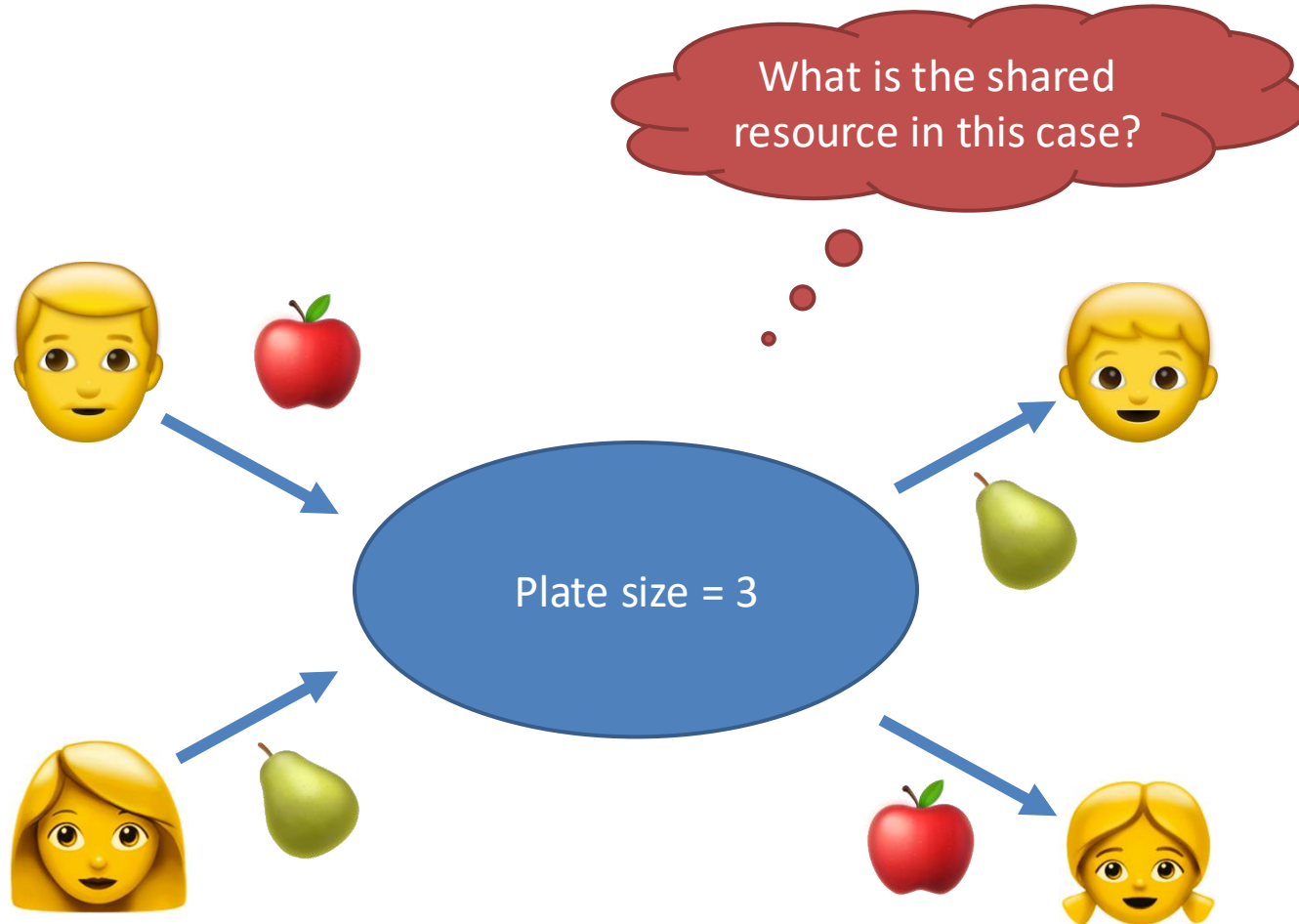


Semaphore

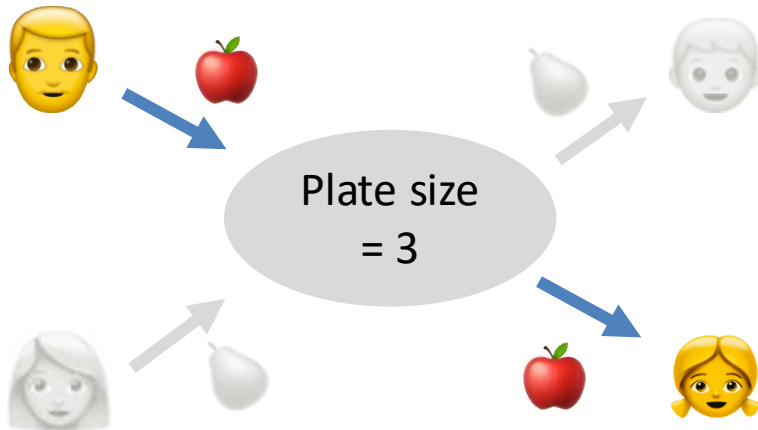
- A semaphore “sem” is a special integer on which only two operations can be performed.
 - DOWN(sem)
 - UP(sem)
- Down operation (P; request):
 - Checks if a semaphore is > 0 , sem--
 - Request one-unit resource and one process enters
 - if a semaphore ≤ 0 , wait and sleep
- Up operation (V; release)
 - sem++
 - Release one-unit resource and one process leaves



Semaphore example



Semaphore example



- Semaphore of apple (s_2):

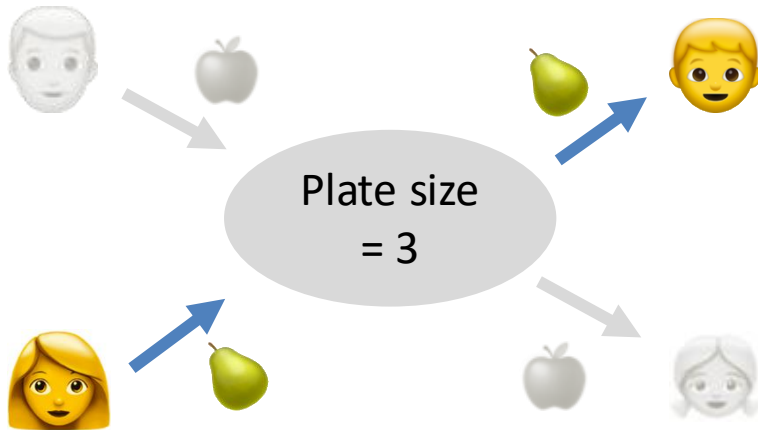
- Daughter: request apple
- Father: release the apple

Father thread:
peel apple
put apple
 $V(s_2)$

Daughter thread:
 $P(s_2)$
get apple
eat apple



Semaphore example



- Semaphore of pear (**s1**):

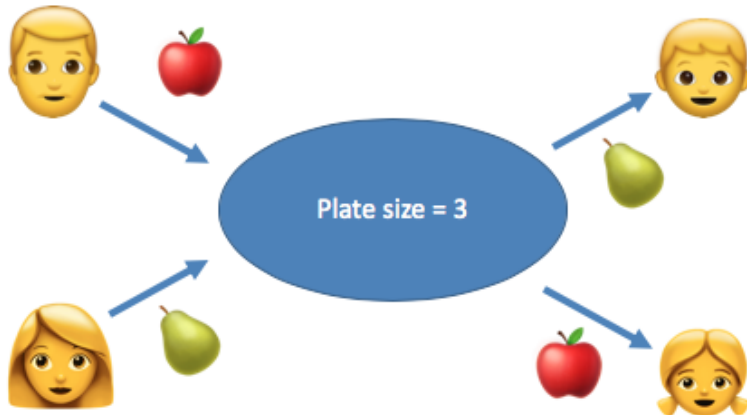
- Son: request pear
- Mother: release the pear

Mother thread:
peel pear
put pear
V(**s1**)

Son thread:
P(**s1**)
get pear
eat pear



Semaphore example



- Semaphore of plate (s_3):
 - Son/Daughter: release the space
 - Father/Mother: request the space

Father thread:
peel apple
 $P(s_3)$
put apple

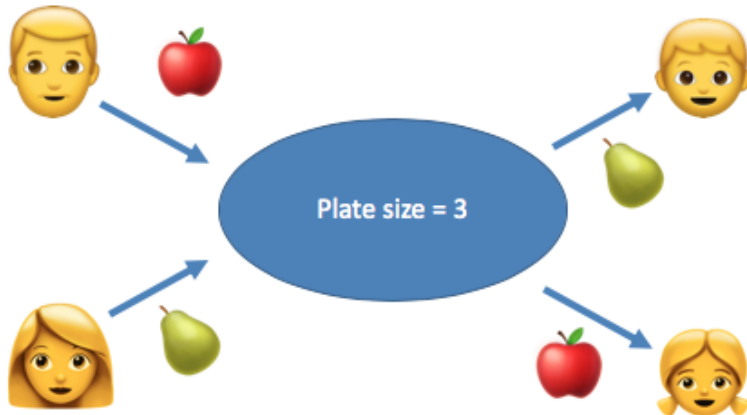
Mother thread:
peel pear
 $P(s_3)$
put pear

Son thread:
get pear
 $V(s_3)$
eat pear

Daughter thread:
get apple
 $V(s_3)$
eat apple



Semaphore example



- Semaphore:

- Son: whether there is pear, $s1$
- Daughter: whether there is apple, $s2$
- Father/Mother: whether there is space, $s3$

Father thread:
peel apple
 $P(s3)$
put apple
 $V(s2)$

Mother thread:
peel pear
 $P(s3)$
put pear
 $V(s1)$

Son thread:
 $P(s1)$
get pear
 $V(s3)$
eat pear

Daughter thread:
 $P(s2)$
get apple
 $V(s3)$
eat apple



Semaphore example

- Semaphore:
 - Son: whether there is pear, **pear**
 - Daughter: whether there is apple, **apple**
 - Father/Mother: whether there is space, remain

Father thread:

peel apple

P(remain)

put apple

V(**apple**)

Daughter thread:

P(**apple**)

get apple

V(remain)

eat apple

```
void *father(void *arg) {
    while(1) {
        sleep(5); //simulate peel apple
        P(s3) sem_wait(&remain);
        sem_wait(&mutex);
        nremain--;
        napple++;
        sem_post(&mutex);
        V(s2) sem_post(&apple);
    }
}
```

```
void *daughter(void *arg) {
    while(1) {
        P(s2) sem_wait(&apple);
        sem_wait(&mutex);
        nremain++;
        napple--;
        sem_post(&mutex);
        V(s3) sem_post(&remain);
        sleep(10); //simulate eat apple
    }
}
```

Semaphore example

```
pi@raspberrypi ~/Downloads> ./semaphore.o
father 🧑 before put apple, remain=3, apple🍏=0, pear🍐=0
father 🧑 after put apple, remain=2, apple🍏=1, pear🍐=0

daughter👧 before eat apple, remain=2, apple🍏=1, pear🍐=0
daughter👧 after eat apple, remain=3, apple🍏=0, pear🍐=0

mother 🧑 before put pear , remain=3, apple🍏=0, pear🍐=0
mother 🧑 after put pear , remain=2, apple🍏=0, pear🍐=1

son 🧑 before eat pear , remain=2, apple🍏=0, pear🍐=1
son 🧑 after eat pear , remain=3, apple🍏=0, pear🍐=0

father 🧑 before put apple, remain=3, apple🍏=0, pear🍐=0
father 🧑 after put apple, remain=2, apple🍏=1, pear🍐=0

mother 🧑 before put pear , remain=2, apple🍏=1, pear🍐=0
mother 🧑 after put pear , remain=1, apple🍏=1, pear🍐=1

daughter👧 before eat apple, remain=1, apple🍏=1, pear🍐=1
daughter👧 after eat apple, remain=2, apple🍏=0, pear🍐=1

father 🧑 before put apple, remain=2, apple🍏=0, pear🍐=1
father 🧑 after put apple, remain=1, apple🍏=1, pear🍐=1

son 🧑 before eat pear , remain=1, apple🍏=1, pear🍐=1
son 🧑 after eat pear , remain=2, apple🍏=1, pear🍐=0

mother 🧑 before put pear , remain=2, apple🍏=1, pear🍐=0
mother 🧑 after put pear , remain=1, apple🍏=1, pear🍐=1

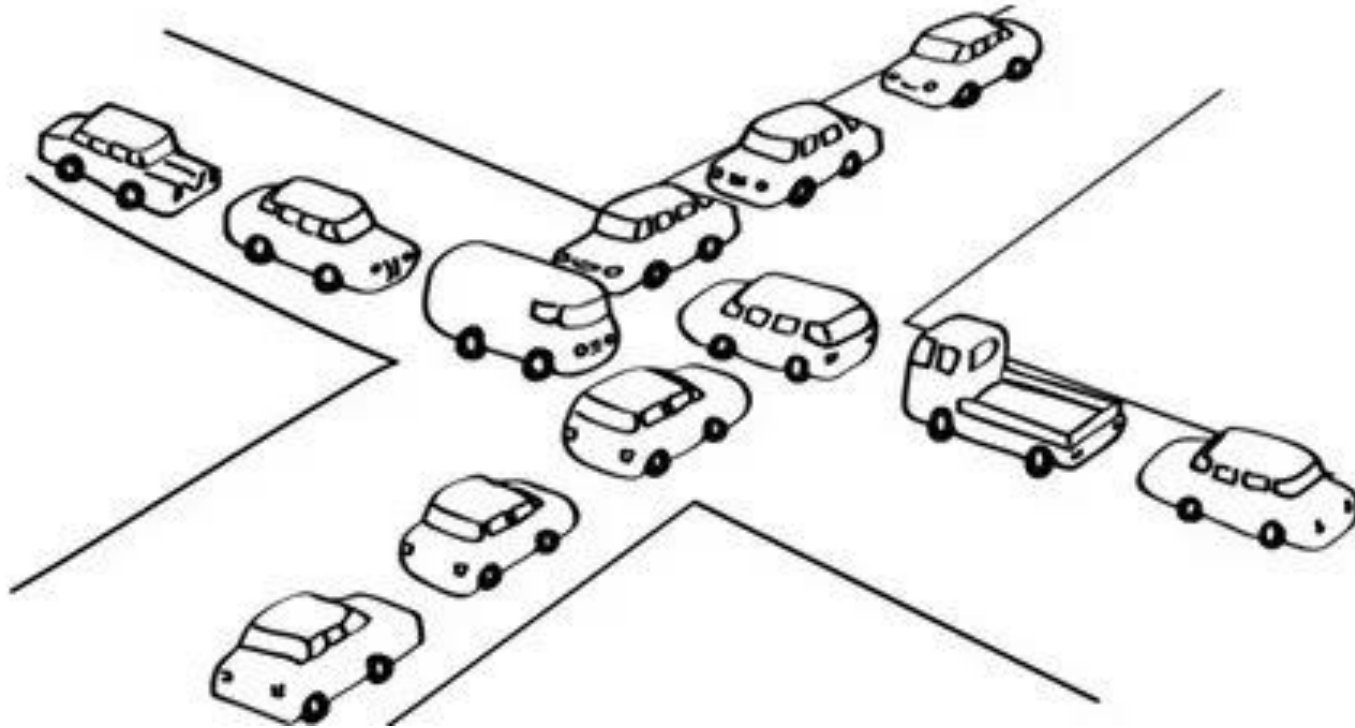
father 🧑 before put apple, remain=1, apple🍏=1, pear🍐=1
father 🧑 after put apple, remain=0, apple🍏=2, pear🍐=1

daughter👧 before eat apple, remain=0, apple🍏=2, pear🍐=1
daughter👧 after eat apple, remain=1, apple🍏=1, pear🍐=1
```

gcc -pthread semaphore.c
-o semaphore.o

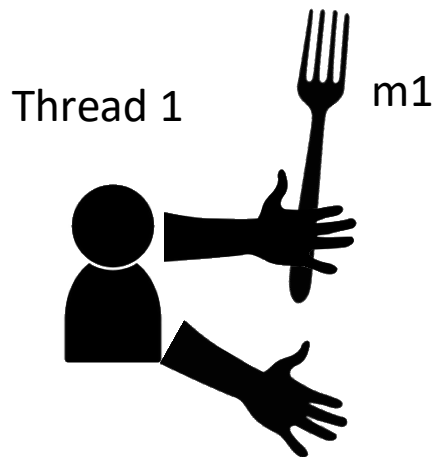
[Demo link](#)

Deadlocks



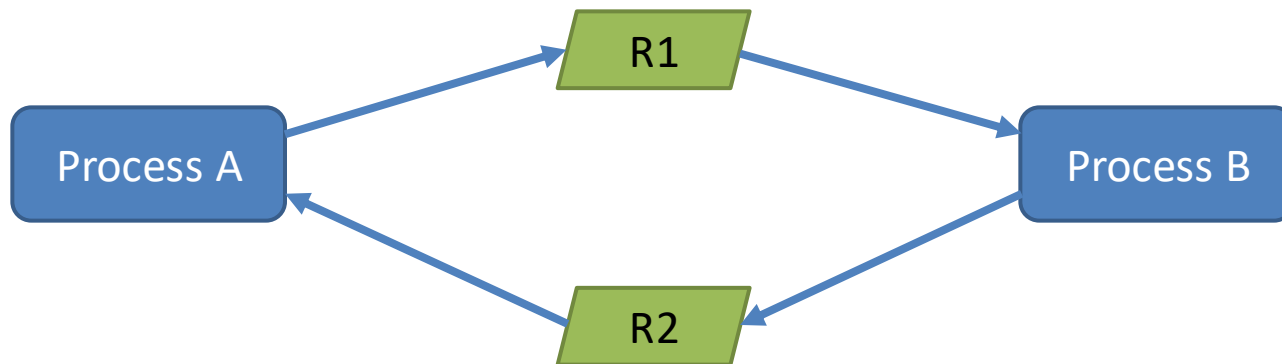
Deadlocks: philosopher dining

- Six people
- Three forks
- Three knives

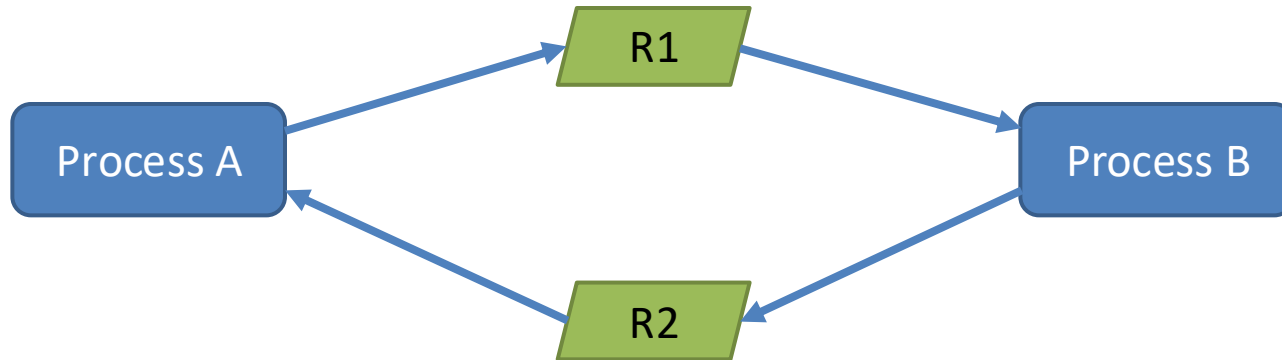


Deadlocks

- When two or more threads stop making progress indefinitely because they are **all waiting for each other** to do something.
 - If process A waits for process B to release a resource, and
 - Process B is waiting for process A to release another resource at the same time.
 - In this case, neither A nor B can proceed because both are waiting for the other to proceed.



Deadlock example



Thread 1

```
pthread_mutex_lock(&R1);  
/* use resource 1 */  
pthread_mutex_lock(&R2);  
/* use resources 1 and 2 */  
do_something();  
pthread_mutex_unlock(&R2);  
pthread_mutex_unlock(&R1);
```



Thread 2

```
pthread_mutex_lock(&R2);  
/* use resource 2 */  
pthread_mutex_lock(&R1);  
/* use resources 1 and 2 */  
do_something();  
pthread_mutex_unlock(&R1);  
pthread_mutex_unlock(&R2);
```



A Joke about Deadlock



Deadlock example: Priority Inversion

[video 1 on youtube](#)

[video 2 on youtube](#)

[story website](#)



1997/07/04 Pathfinder
—> Mars

